

Distributed Systems - 17_11

Let's see the algorithm of Paxos and the proof.

Here is the code for the proposer:

Algorithm 1 Paxos — Proposer p

```
1: Constants:  
2:  $A, n$ , and  $f$ .           { $A$  is the set of acceptors.  $n = |A|$  and  
    $f = \lfloor (n - 1)/2 \rfloor$ .}  
3: Init:  
4:  $crnd \leftarrow -1$            {Current round number}  
5: on  $\langle \text{PROPOSE}, val \rangle$   
6:    $crnd \leftarrow \text{pickNextRound}(crnd)$   
7:    $cval \leftarrow val$   
8:    $P \leftarrow \emptyset$   
9:   send  $\langle \text{PREPARE}, crnd \rangle$  to  $A$   
10: on  $\langle \text{PROMISE}, rnd, vrnd, vval \rangle$  with  $rnd = crnd$  from  
    acceptor  $a$   
11:    $P \leftarrow P \cup (vrnd, vval)$   
12: on event  $|P| \geq n - f$   
13:    $j = \max\{vrnd : (vrnd, vval) \in P\}$   
14:   if  $j \geq 0$  then  
15:      $V = \{vval : (j, vval) \in P\}$   
16:      $cval \leftarrow \text{pick}(V)$            {Pick proposed value  $vval$  with  
      largest  $vrnd$ }  
17:   send  $\langle \text{ACCEPT}, crnd, cval \rangle$  to  $A$ 
```

f is the amount of failures we can tolerate.

$crnd$ is the current round number.

When a proposer wants to propose, we trigger the handler. The handler who proposes can only propose once, it's a one-shot algorithm.

I set the current round number to the next round number assigned to me.

P is a set in which I collect the response from the acceptors. We then trigger the prepare phase, sending the messages to all acceptors in A .

Here is the acceptor code:

Algorithm 2 Paxos — Acceptor a

```
1: Constants:  
2:  $L$  {Set of learners}  
  
3: Init:  
4:  $rnd \leftarrow -1$   
5:  $vrnd \leftarrow -1$   
6:  $vval \leftarrow -1$   
  
7: on  $\langle \text{PREPARE}, prnd \rangle$  with  $prnd > rnd$  from proposer  $p$   
8:    $rnd \leftarrow prnd$   
9:   send  $\langle \text{PROMISE}, rnd, vrnd, vval \rangle$  to proposer  $p$   
  
10: on  $\langle \text{ACCEPT}, i, v \rangle$  with  $i \geq rnd$  from proposer  $p$   
11:    $rnd \leftarrow i$   
12:    $vrnd \leftarrow i$   
13:    $vval \leftarrow v$   
14:   send  $\langle \text{LEARN}, i, v \rangle$  to  $L$ 
```

An acceptor has a round number that is -1, we can accept any value, then we have a pair $(value, round)$ that we have accepted.

The PREPARE message sent by the proposer reaches the acceptor triggering the handler, and we check if the proposed round number is higher than the rnd we saved. If yes, we update our promise with the new value, and send a PROMISE message to the proposer p (note that PROMISE is the ACK sent by the acceptor).

Let's go back to the proposer code, which will receive the promise back. The promise has to be for the current round number, and we update the P set with the ACK from an acceptor. Once we reach a majority of responses, so when $|P| \geq n - f$, we trigger another handler. I will then scan the set P , and take the value associated with the greatest round number. If that element has a round number different from -1, then something has already been accepted by an acceptor, and so we have to modify our value. Otherwise, we send our value. In case of $j \neq -1$, we take the value associated to j , and modify my value with the one picked from the set of P . We send now our ACCEPT message with the value and round number.

Notice that our algorithm does not use a NACK, in fact this algorithm uses an implicit NACK, so that instead of sending a NACK, we do not reply, and that

counts as a NACK. We have a timeout set, once the timeout is reached and we've received no answer, we trigger again the Propose handler, picking a new assigned round number.

Let's go back to the acceptor code.

Here we'll deal with the ACCEPT message. We check that the round number sent is $\geq$ than the round number we promised. If someone is sending me an accept for an old round number, I won't accept the value. In case $i \geq rnd$, we set our value, as well as the round number, to the one received, and send a message to the learner processes, LEARN, with the value they have to learn.

So, here is the code of the learner:

Algorithm 3 Paxos — Learner l

```
1: Init:  
2:  $V \leftarrow \emptyset$   
3: on  $\langle \text{LEARN}, (i, v) \rangle$  from acceptor  $a$   
4:    $V \leftarrow V \uplus (i, v)$   
5: on event  $\exists i, v : |\{(i, v) : (i, v) \in V\}| \geq n - f$   
6:    $v$  is chosen
```

A learner has a multi-set of values V , initially set to $\emptyset$. What is a multi-set? In a set, elements are unique, while in a multi-set we can have repetitions of elements, along side the multiplicity for a value. When it receives a message from an acceptor, I add the message to a set.

Then, when there exists a value v such that the number of times it appears is $\geq n - f$ then we have that v is chosen.

In the best case, what is the message delay? It's 3, as we have the prepare, then the acks, and proposal. 4 if we count also the messages to the learners.

Let's now see the proof:

We'll just prove the agreement property.

We say that if an acceptor a_i accepts to a proposal $\langle num = r, val \rangle$ we say that a_i voted for val at round r .

The property we want to show is this one: If an acceptor a has voted for value v at round i , then no value v' different from v can be chosen in a previous round $j < i$.

If we go to a previous round, we can't see a different value being chosen.

The proof is by induction on round number. It's based on a observation: if an acceptor vote for a certain value at a certain round, then there is a proposer that collects a set of acks from a quorum of acceptors for its prepare.

Let's take $r = 0$ as the base case. In $r = 0$ the claim is correct, there are no $r < 0$. It's not possible there were any value chosen before as we are the first to send. The inductive hypothesis. We say that the claim is correct up to a round number $i - 1$. Starting from this, we have to prove that it's correct for the next round $r = i$. If a proposer is sending a proposal for this round number, then we received a set of promises that allowed us to send a value v .

Let's do an example in a real system.

Let us investigate round $r=i$. Let $\langle r=j, v \rangle$ the vote for the highest round seen in the set S (recall the $i > j$).

Example with $j=4$, and $i=8$
 $Q:\{A1, A2, A3\}$
 The - means that the Acceptor promised p , to not vote for any proposal in that round. Recall, that if **a** sends us a response to our prepare to round i , it will not vote for any other round less than i .
 The ? Means that we do not know, since $A4$ is not in Q .

	$r=4$	$r=5$	$r=6$	$r=7$
A1	v	-	-	-
A2	-	-	-	-
A3	v	-	-	-
A4	?	?	?	?

Is it possible that a different value from the one we proposed in $r = 5, 6, 7$ is accepted? No, as there is no majority for those. If an acceptor votes for a round, no value can be chosen in new rounds.

When we receive the messages from the acceptor, $r = 4$ is the highest round number. It's not possible that we find two different values. Then I, as a proposer, take value v and send it again. However, if we find $r = 4$ then the acceptor has voted for that value and no other before, in fact the inductive hypothesis holds, if an acceptor has voted a value, then it's not possible that in a previous round the value was chosen.

This proof shows that there is agreement for Paxos. We're not discussing the termination. However, to show this, we need to assume the system becomes synchronous. What is the scenario that kills our termination goal? It's the scenario in which proposers kill each other value. How do we fix this? We have to assume

that the system is eventually synchronous. Also, the period of synchrony is long enough to have Paxos terminates.

In real life, we could have a leader election for the proposer.

Let's see a question:

Let us suppose that we have a run of paxos where proposer P_1 is able to make $n/2-1$ acceptors vote its proposal at a round j ($a_1, a_2, \dots, a_{n/2-1}$), and let us say that its proposal has value 3,

And where process P_2 makes the other $n/2-1$ acceptors vote its proposal at round $j+1$ ($a_{n/2+1}, \dots, a_n$) and its proposal has value 42.

One acceptor $a_{n/2}$ does not receive any proposal.

It is possible that a different proposer, on a round $j+k$, makes acceptors decide on value 7?

Two processes have the value voted for two minorities. One acceptor does not receive anything.

If we start after that round, we, as a new proposer, we'll get a majority of values before proposing a new value. So in that majority we have to receive at least one among the value 3 or 42, so we can't send a new value. If we receive a mix of them, then we have to propose 42 (as it has the highest round number).

Is Paxos used in real life? Yes, one famous system that uses Paxos is the chubby-system, implemented in Google. Chubby is a distributed lock manager, which is a system in which we can create a lock with a certain name, and take it and release it.

Let's now see some exercises for Paxos.

In Paxos, suppose that a cluster contains 5 servers and 3 of them have accepted proposal 6 with value X. Once this has happened, is it possible that any server in the cluster could accept a different value Y?

Explain your answer:

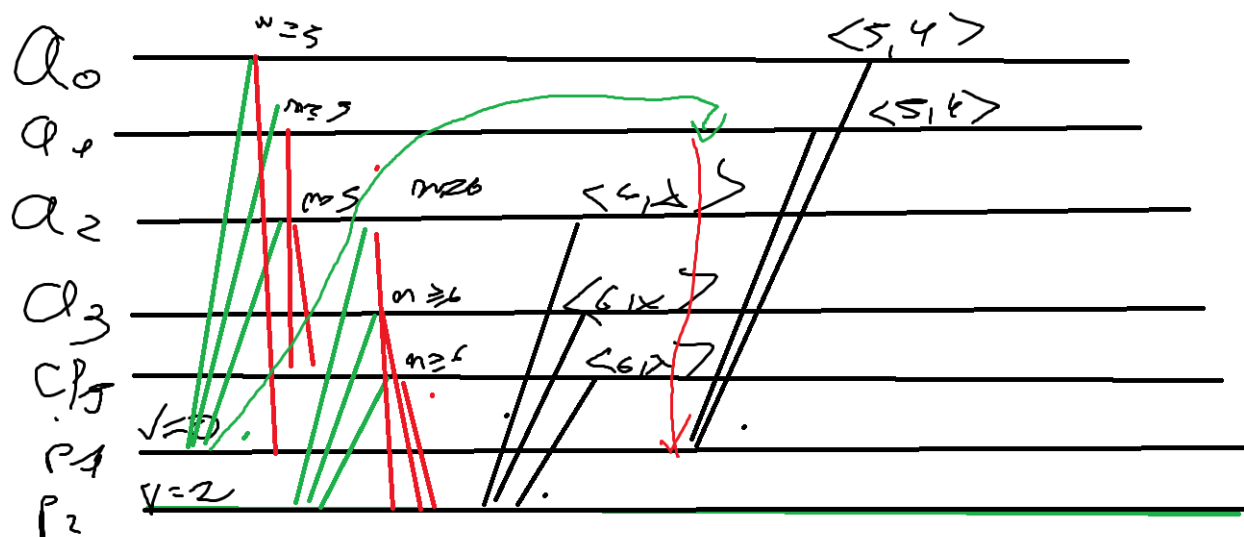
-If possible shows an execution;

-If not possible describe the rules of the algorithm that prevent such behaviour.

We have that at a certain time t three acceptors will have $(6, X)$ in memory. It's impossible that if, given (r, Y) , a value with round number r is accepted.

What happens if $r < 6$? For example we have $r = 5$. a_4 and a_5 could have -1 still as r , and so they would accept it.

Let's see an execution.



Let's see another exercise:

In Paxos, we have seen that each participant with id k (k in $\{1, 2, \dots, n\}$), number its prepare requests using a number in the set $m \cdot n + k$ with m natural number.

That is participant 1 is allowed to prepare $\{1, n+1, 2n+1, 3n+1, \dots\}$.

Participant 2 is allowed to prepare $\{2, n+2, 2n+2, \dots\}$.

....

Participant n prepares $\{n, 2n, 3n, 4n, \dots\}$.

Suppose we let each participant timestamps its prepare requests with the pair (ID, ts) (where ts is an increasing counter) (Example $(1,0), (1,1), (1,2), \dots$ for participant 1), and we use the following rule $(ID', ts') > (ID, ts)$ if $ID' > ID$ or $ID' = ID$ and $ts' > ts$.

Discuss if the properties of safety and liveness of Paxos are impacted.

Is the safety impacted? No, as for the fact that round numbers are still unique.

Also, we have total order on the round.

Is the liveness violated? Let's say we have p_0, p_1, p_2 , p_2 will always have priority.

This priority will be a problem, in fact once a majority goes to $(2, 0)$, no other

processes will get majority. However, if p_2 crashes before proposing, is p_1 able to

prepare a new request? No, as he will only receive a minority (assuming that p_1

received the majority of ACKS, locking acceptors). So, we are violating liveness as

here processes will never decide.

Let's do one other exercise:

Consider the modified implementation of paxos in the following slides. Discuss the Safety and Liveness properties of paxos. Specifically, if a property holds discuss why, if it does not, show a counterexample.

Algorithm 1 Paxos — Proposer p

```
1: Constants:
2:  $A, n$ , and  $f$ .      { $A$  is the set of acceptors.  $n = |A|$  and
    $f = \lfloor (n - 1)/2 \rfloor$ .}
3: Init:
4:  $crnd \leftarrow -1$       {Current round number}
5: on  $\langle \text{PROPOSE}, val \rangle$ 
6:    $crnd \leftarrow \text{pickNextRound}(crnd)$ 
7:    $cval \leftarrow val$ 
8:    $P \leftarrow \emptyset$ 
9:   send  $\langle \text{PREPARE}, crnd \rangle$  to  $A$ 
10: on  $\langle \text{PROMISE}, rnd, vrnd, vval \rangle$  with  $rnd = crnd$  from
    acceptor  $a$ 
11:    $P \leftarrow P \cup (vrnd, vval)$ 
12: on event  $|P| \geq n - f$ 
13:    $j = \max\{vrnd : (vrnd, vval) \in P\}$ 
14:   if  $j \geq 0$  then
15:      $V = \{vval : (j, vval) \in P\}$ 
16:      $cval \leftarrow \text{pick}(V)$       {Pick proposed value  $vval$  with
    largest  $vrnd$ }
17:   send  $\langle \text{ACCEPT}, crnd, cval \rangle$  to  $A$ 
```

There is an
implicit timeout that re-calls
propose

Algorithm 2 Paxos — Acceptor a

```
1: Constants:  
2:  $L$  {Set of learners}  
  
3: Init:  
4:  $rnd \leftarrow -1$   
5:  $vrnd \leftarrow -1$   
6:  $vval \leftarrow -1$   
  
7: on  $\langle \text{PREPARE}, prnd \rangle$  with  $prnd > rnd$  from proposer  $p$   
8:    $rnd \leftarrow prnd$   
9:   send  $\langle \text{PROMISE}, rnd, vrnd, vval \rangle$  to proposer  $p$   
10: on  $\langle \text{ACCEPT}, i, v \rangle$  from proposer  $p$   
11:    $rnd \leftarrow i$   
12:    $vrnd \leftarrow i$   
13:    $vval \leftarrow v$   
14:   send  $\langle \text{LEARN}, i, v \rangle$  to  $L$ 
```

This code does not
send nack. This is ok
if there is a timeout on
proposers.

Algorithm 3 Paxos — Learner l

```
1: Init:  
2:  $V \leftarrow \emptyset$   
  
3: on  $\langle \text{LEARN}, (i, v) \rangle$  from acceptor  $a$   
4:    $V \leftarrow V \uplus (i, v)$   
  
5: on event  $\exists i, v : |\{(i, v) : (i, v) \in V\}| \geq n - f$   
6:    $v$  is chosen
```

In the modified version, we have that the acceptor accepts any proposal, thus removing the acceptance based on promise.

What happens to:

- integrity
- validity
- agreement
- termination assuming the synchronicity

Are we violating validity? We're not changing how messages are handled, so we are not inventing a value. Can we violate integrity? No as well.

Let's see termination. To remove the termination, I should restrict the algorithm more. If I remove the predicate, we have the acceptor send more message than

normal, and we would not be violating it.

What about agreement? We are violating it. To satisfy agreement, we have to have that once a value is chosen, in the future rounds we can't accept a new value. For a proposal with round number smaller, in the original algorithm we kill that proposal as the one with higher round number has already been chosen. If in this algorithm we have $r = 10$ chosen, and $r = 9$ is still in the system, it's possible that we accept $r = 9$ even if we are at round 10.

Let's see the execution:

